

AN INDUSTRIAL INTERNSHIP PROJECT REPORT ON

FORECASTING OF SMART CITY TRAFFIC PATTERNS

A Time-Series Vehicle Volume Forecasting System using XGBoost Regression

Submitted in partial fulfilment of the requirements of the
Industrial Internship Program by Upskill Campus (USC) & The IoT Academy,
in association with Industry Partner UniConverge Technologies Pvt. Ltd. (UCT)

Prepared by
Purusottam Dash

B.Tech, Computer Science Engineering (AI & ML)
Internship Duration: [15/06/2026] – [07/07/2026]

Executive Summary

This report presents the work completed during a six-week Industrial Internship Program conducted by Upskill Campus and The IoT Academy in collaboration with UniConverge Technologies Pvt. Ltd. (UCT).

The assigned project, titled "Forecasting of Smart City Traffic Patterns", involved building a machine learning pipeline to predict hourly vehicle counts at four city junctions using roughly twenty months of historical traffic data.

An XGBoost Regressor was trained on time-derived features (year, month, day, hour, day-of-week, weekend flag) and the junction identifier, achieving a validation R^2 score of 0.948 and an RMSE of 4.60 vehicles/hour.

This internship provided practical exposure to real-world data wrangling, time-series feature engineering, gradient boosting models, and end-to-end ML pipeline design — an overall valuable and rewarding experience.

Table of Contents

1. Preface	3
2. Introduction	4
2.1 About the Internship Program.....	4
2.2 About UniConverge Technologies Pvt. Ltd. (UCT)	4
2.3 About Upskill Campus (USC) and The IoT Academy	4
2.4 Objectives of the Internship Program.....	4
2.5 Reference	4
2.6 Glossary.....	5
4. Existing and Proposed Solution	6
4.1 Existing Approaches and Their Limitations	6
4.2 Proposed Solution	6
4.3 Value Addition.....	6
4.4 Code and Report Submission.....	6
5. Proposed Design / Model.....	7
5.1 High-Level Diagram	7
5.2 Low-Level Diagram (Data & Processing Pipeline)	7
5.3 Interfaces	8
6.1 Test Plan / Test Cases	9
6.2 Test Procedure	9
6.3 Performance Outcome	9

1. Preface

This report presents a summary of the work completed during my six-week Industrial Internship Program, carried out under the joint aegis of Upskill Campus (USC) and The IoT Academy, in association with the industry partner UniConverge Technologies Pvt. Ltd. (UCT), as part of an AICTE-recognised industrial internship framework.

Industrial internships play a crucial role in bridging the gap between classroom learning and real engineering practice. As a B.Tech Computer Science Engineering student specialising in Artificial Intelligence and Machine Learning, I have worked on academic projects and coursework involving deep learning and computer vision, but this internship gave me my first structured opportunity to take a raw, messy, real-world dataset all the way through to a validated and deployable predictive model within a fixed, industry-style timeline.

My assigned problem statement was “Forecasting of Smart City Traffic Patterns” — building a system to predict hourly vehicle counts at four city junctions using close to two years of historical traffic data, with the completed model, code, and this report to be delivered within six weeks.

The program was structured around weekly milestones: understanding the problem statement and dataset, exploratory data analysis, feature engineering, model selection and training, performance evaluation, and finally documentation and reporting. Regular guidance from my internship mentor and structured checkpoints helped keep the project on track.

Working through this project deepened my understanding of applied time-series feature engineering, gradient boosting algorithms, and the discipline of validating a model honestly before trusting its predictions. It also reinforced a lesson that I believe is broadly true in applied ML: most of the performance gain in this project came from asking the right questions of the data and engineering sound features, not from exotic modelling tricks.

I would like to thank my institute, ITER, Siksha 'O' Anusandhan (SOA) University, Bhubaneswar, my internship mentor and the Upskill Campus / UCT program team for their guidance and support throughout this internship. I am also grateful to my faculty coordinators and peers for their continuous encouragement.

To my juniors and peers who will take up similar internships: spend real time understanding your dataset and the business problem behind it before jumping into modelling — it pays off far more than premature hyperparameter tuning.

2. Introduction

2.1 About the Internship Program

This internship was offered under the joint industrial training initiative of Upskill Campus and The IoT Academy, in collaboration with UniConverge Technologies Pvt. Ltd. (UCT) as the industry partner. The program is designed to give engineering students hands-on, industry-style project experience aligned with the AICTE internship framework, with each intern assigned a real problem statement to solve independently within a fixed timeline, under mentorship.

2.2 About UniConverge Technologies Pvt. Ltd. (UCT)

UniConverge Technologies Pvt. Ltd., established in 2013, works in the Digital Transformation domain, building industrial solutions with a strong focus on sustainability and return on investment. UCT leverages cutting-edge technologies including the Internet of Things (IoT), Cyber Security, Cloud Computing (AWS, Azure), Machine Learning, communication technologies (4G/5G/LoRaWAN), and full-stack software development to build its product lines, which include an IoT application platform, a smart-factory monitoring platform, LoRaWAN-based smart city and metering solutions, and predictive maintenance systems for industrial machinery.

The problem statement for this internship — forecasting traffic volumes for smart cities — sits squarely within UCT's smart-city and IoT-driven analytics focus, applying the same data-driven, ML-based approach that the company uses across its industrial product lines.

2.3 About Upskill Campus (USC) and The IoT Academy

Upskill Campus is a career development platform that delivers personalised, affordable, and measurable skill-building programs, facilitating the smooth execution of the complete internship process alongside its academy and industry partners. The IoT Academy is the EdTech division associated with this ecosystem, running long-format executive certification programs in collaboration with premier institutes, through which structured internships such as this one are delivered to engineering students.

2.4 Objectives of the Internship Program

- To gain practical, hands-on experience of working on a real industry-style problem statement.
- To apply classroom knowledge of machine learning to solve a real-world forecasting problem end-to-end.
- To build a portfolio-ready project that improves job and internship prospects in the AI/ML domain.
- To gain a deeper, applied understanding of time-series data handling and gradient boosting models.
- To strengthen personal skills such as independent problem solving, time management, and technical documentation.

2.5 Reference

[1] Smart City Traffic Patterns dataset, provided as the internship problem statement (originally sourced from a public traffic-forecasting dataset covering four city junctions, Nov 2015 – Jun 2017).

[2] XGBoost Documentation — <https://xgboost.readthedocs.io>

[3] Scikit-learn Documentation — <https://scikit-learn.org>

[4] Project Code Repository — <https://github.com/purusottamdash0-collab/Forecasting-of-Smart-city-Traffic-Patterns->

2.6 Glossary

Term	Meaning
RMSE	Root Mean Squared Error — average magnitude of prediction error, in the same unit as the target (vehicles/hour).
R ² Score	Coefficient of determination — proportion of variance in the target explained by the model (1.0 = perfect fit).
XGBoost	Extreme Gradient Boosting — an ensemble of decision trees built sequentially to correct prior errors.
Feature Engineering	The process of deriving new, informative input variables (e.g. Hour, DayOfWeek) from raw data.
Validation Set	A held-out portion of training data used to estimate model performance on unseen data.
Feature Importance	A model-derived score indicating how much each input feature contributed to predictions.
EDA	Exploratory Data Analysis — visual/statistical inspection of data before modelling.

3. Problem Statement

Traffic congestion is one of the most persistent challenges faced by growing smart cities. City traffic authorities need the ability to anticipate vehicular load at key junctions in advance, so that signal timing, lane management, and resource allocation can be planned proactively rather than reactively.

The problem statement assigned for this internship was: given approximately twenty months of hourly vehicle-count history recorded at four city junctions (November 2015 – June 2017), build a machine learning model that can forecast the number of vehicles expected to pass through each junction over a subsequent, previously unseen four-month horizon (July – October 2017), using only time-derived and junction-identifying information available at prediction time — i.e. without access to future traffic counts themselves.

Key characteristics of the assigned dataset that shaped the approach:

- Training data: 48,120 hourly records across 4 junctions, with columns DateTime, Junction, Vehicles, and ID.
- Junction 4 has substantially less history (4,344 records) than Junctions 1–3 (14,592 records each) — an imbalanced, “cold-start” style junction.
- Test data: 11,808 records (2,952 per junction) covering a genuinely unseen future 4-month window.
- Vehicle counts range from 1 to 180 per hour, with a mean of ≈ 22.8 and standard deviation of ≈ 20.75 , indicating a right-skewed distribution with occasional high-traffic spikes.
- No missing values were present in the raw training data.
- The dataset was delivered inside a doubly-nested ZIP archive with inconsistent internal file names, adding a real data-wrangling challenge before modelling could even begin.

4. Existing and Proposed Solution

4.1 Existing Approaches and Their Limitations

- Fixed-timer / rule-based traffic signal control — does not adapt to actual, time-varying traffic load and cannot anticipate future congestion.
- Classical statistical time-series models (ARIMA/SARIMA) fitted per junction — assume linear, stationary behaviour, struggle to incorporate exogenous or categorical signals cleanly, and require a separate model per junction, which is costly to maintain as a city scales to hundreds of junctions.
- Simple moving-average / exponential-smoothing baselines — fail to capture the interaction between daily and weekly seasonality (e.g. rush hour effects differing between weekdays and weekends).
- Sensor-triggered, rule-based congestion alerts — inherently reactive rather than predictive, so they cannot support proactive planning.

4.2 Proposed Solution

The proposed solution uses a single supervised regression model — an XGBoost Regressor — trained jointly across all four junctions, with the junction identifier included as a model feature alongside time-derived features engineered from the DateTime column (Year, Month, Day, Hour, DayOfWeek, and an IsWeekend flag). Because gradient-boosted trees can learn non-linear interactions and junction-specific effects automatically through their splits, a single unified model is able to generalise across junctions instead of requiring one bespoke model per location.

4.3 Value Addition

- A single model serves all junctions, substantially lowering maintenance overhead compared to per-junction statistical models.
- Automatically captures non-linear seasonal interactions (e.g. rush-hour × weekday effects) without manual feature crossing.
- Training and inference are fast, making the approach suitable for scheduled batch forecasting in a production pipeline.
- Feature importances are directly interpretable, helping build trust with traffic planners who are not ML specialists.
- Predictions are post-processed — clipped at zero and rounded to the nearest integer — to respect the physical constraint that vehicle counts can never be negative or fractional.

4.4 Code and Report Submission

Code Repository (GitHub): <https://github.com/purusottamdash0-collab/Forecasting-of-Smart-city-Traffic-Patterns->

5. Proposed Design / Model

This section describes the design flow of the solution, from raw historical data to a validated forecasting pipeline capable of producing predictions for unseen future periods.

5.1 High-Level Diagram



Figure 1: High-Level System Architecture

At a high level, historical hourly traffic records from all four junctions are cleaned and transformed into model-ready features. These features are consumed by an XGBoost regression model, which produces a predicted vehicle count for each future hour and junction. These predictions can then feed into a downstream traffic management or decision-support system for proactive signal timing and congestion planning.

5.2 Low-Level Diagram (Data & Processing Pipeline)

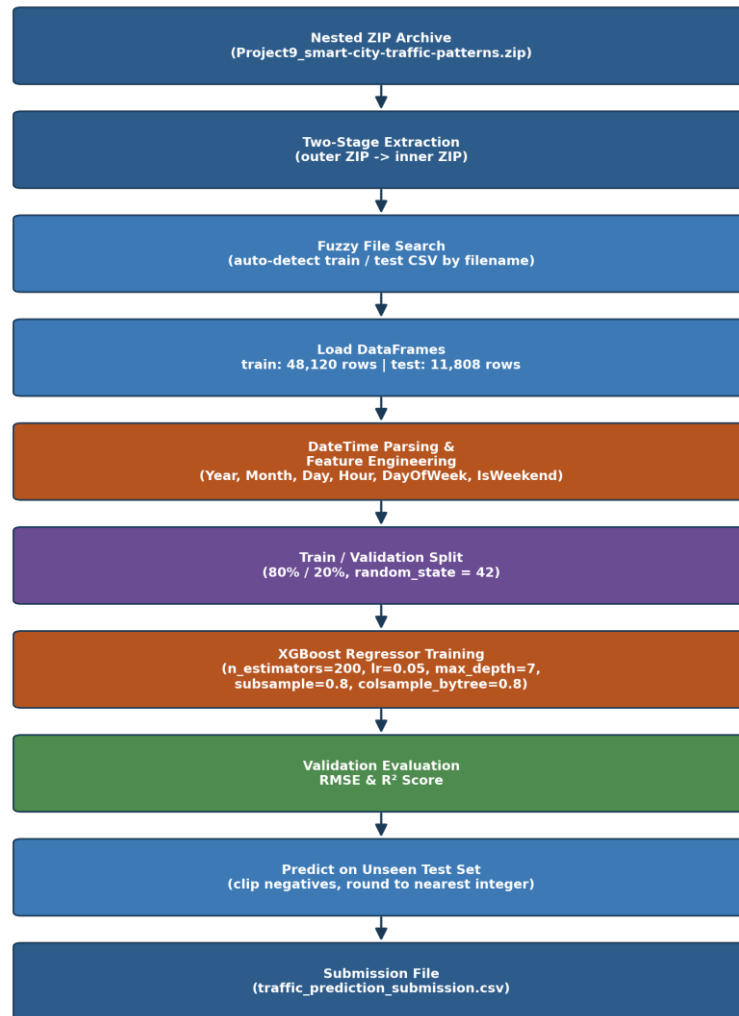


Figure 2: Detailed Data and Processing Pipeline

The dataset arrives as a doubly-nested ZIP archive; the pipeline first performs a two-stage extraction, then a fuzzy filename search to reliably auto-detect the training and test CSV files regardless of their exact naming. The `DateTime` column is then parsed to derive `Year`, `Month`, `Day`, `Hour`, `DayOfWeek`, and `IsWeekend` features. An 80/20 train-validation split (`random_state = 42`) is used for model development. The XGBoost Regressor is configured with 200 estimators, a learning rate of 0.05, a maximum tree depth of 7, and row/column subsampling of 0.8 each — settings chosen to balance model capacity against overfitting risk on a moderately sized tabular dataset. After validation (RMSE and R^2), the same trained model is used to predict on the genuinely unseen test set, with predictions clipped at zero and rounded before being written to a submission file.

5.3 Interfaces

- Input interface: CSV files with columns `DateTime`, `Junction`, `Vehicles`, `ID` (training) and `DateTime`, `Junction`, `ID` (test / inference).
- Output interface: A CSV submission file with columns `ID` and predicted `Vehicles`.
- Internal interfaces: pandas DataFrames pass data between the preprocessing and modelling stages; the model itself follows the standard scikit-learn-compatible `fit()`/`predict()` interface exposed by XGBoost's sklearn API.

- Runtime environment: Python 3 on Google Colab / Jupyter Notebook, using pandas, numpy, matplotlib, seaborn, scikit-learn, and xgboost.

6. Performance Test

This section evaluates why the proposed model is fit for real deployment rather than being a purely academic exercise — by identifying the key performance constraints, describing how they were addressed in the design, and reporting the actual test results obtained.

The primary constraints identified for this problem were prediction accuracy (how close forecasts are to actual traffic volumes) and generalisation (performance on the genuinely unseen four-month test horizon, not just on data resembling the training period).

6.1 Test Plan / Test Cases

#	Test Case	Expected Result
1	Nested ZIP archive extraction	Both train and test CSV files are successfully retrieved.
2	Fuzzy filename search for train/test files	Correct file paths are auto-detected regardless of exact naming.
3	DateTime feature extraction	Year, Month, Day, Hour, DayOfWeek, IsWeekend values match the source timestamp.
4	Model training completion	XGBoost training completes over 200 boosting rounds without error.
5	Validation accuracy threshold	R^2 score exceeds 0.90 and RMSE is well below the target's standard deviation.
6	Physically valid predictions	All predicted vehicle counts are non-negative integers.
7	Submission file schema	Output CSV contains ID and Vehicles columns with 11,808 rows, matching the test set.

6.2 Test Procedure

An 80/20 random split (`random_state = 42`) of the 48,120-row training set was held out for validation. The XGBoost Regressor was trained on the remaining 80% and evaluated on the held-out 20% using RMSE and R^2 . Feature-importance and residual diagnostic plots (actual-vs-predicted snapshot, correlation heatmap, and scatter plot) were then generated to sanity-check that the model was learning genuine seasonal and location-based signal rather than noise, before the same trained model was applied to the true, unseen test set to produce final forecasts.

6.3 Performance Outcome

Metric	Value	Interpretation
RMSE	4.6015 vehicles/hour	Average prediction error is small relative to the target's standard deviation (≈ 20.75).
R^2 Score	0.9480	The model explains $\approx 94.8\%$ of the variance in hourly vehicle counts on held-out data.

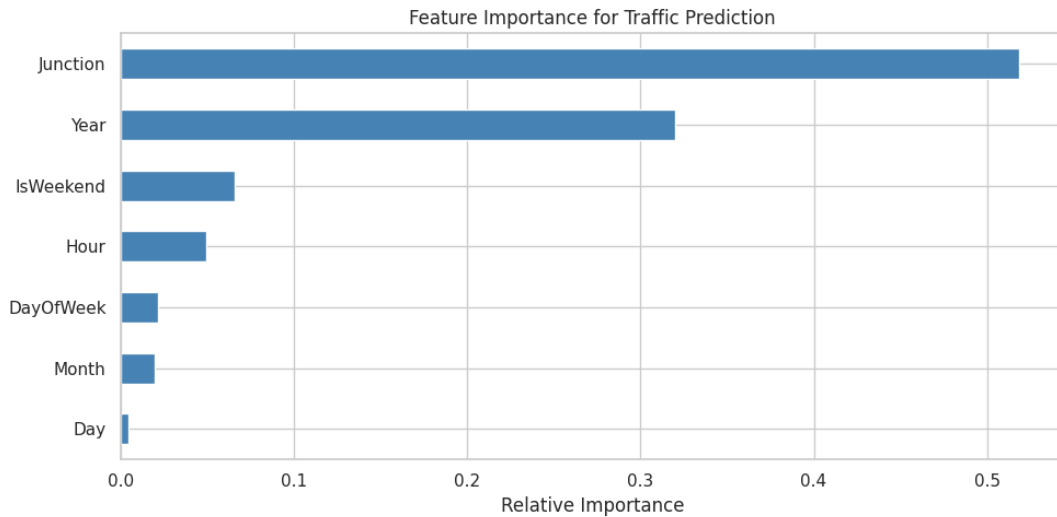


Figure 3: Feature Importance for Traffic Prediction

Junction and Year dominate the model's feature importance, indicating that location-specific baseline load and the long-term growth trend across the two-year window carry the strongest signal. IsWeekend and Hour contribute a moderate, expected seasonal effect, while DayOfWeek, Month, and Day contribute comparatively little on their own — though the tree model still exploits their interactions with the more important features.

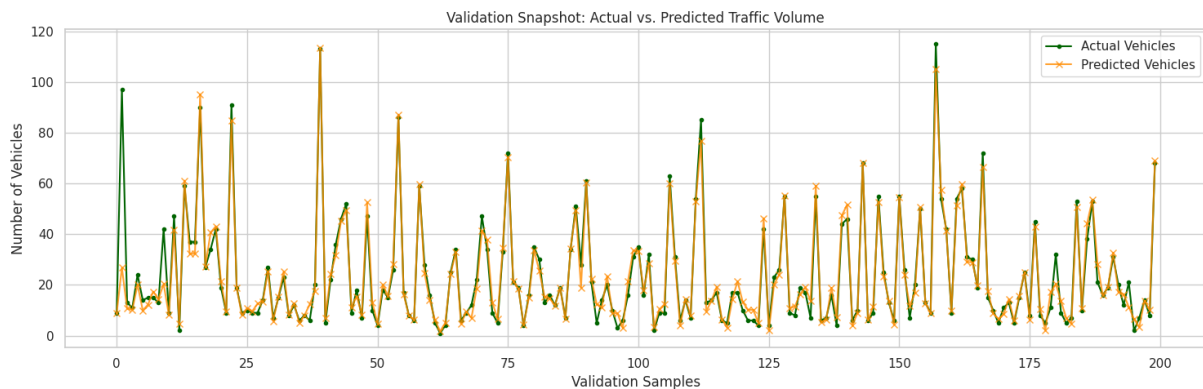


Figure 4: Validation Snapshot — Actual vs. Predicted Traffic Volume

Over a 200-sample validation snapshot, predicted vehicle counts (orange) closely track actual counts (green), including sharp peaks and troughs, confirming that the model captures both the general level and the volatility of traffic reasonably well.

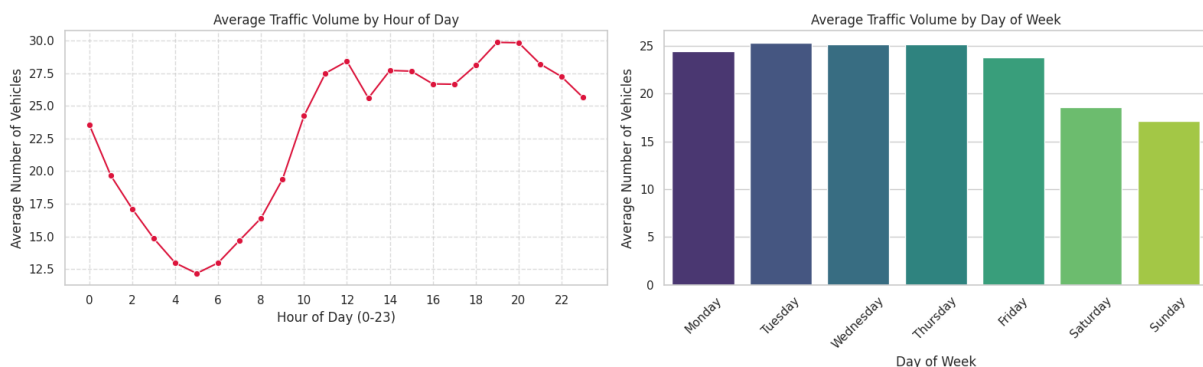


Figure 5: Average Traffic Volume by Hour of Day and Day of Week

Traffic volume follows a clear daily rhythm — dipping to its lowest point around 4–5 AM and rising to a broad evening peak between roughly 7 PM and 8 PM — and a clear weekly rhythm, with weekday traffic (Monday–Friday) consistently higher than weekend traffic (Saturday–Sunday), consistent with typical commuter behaviour.

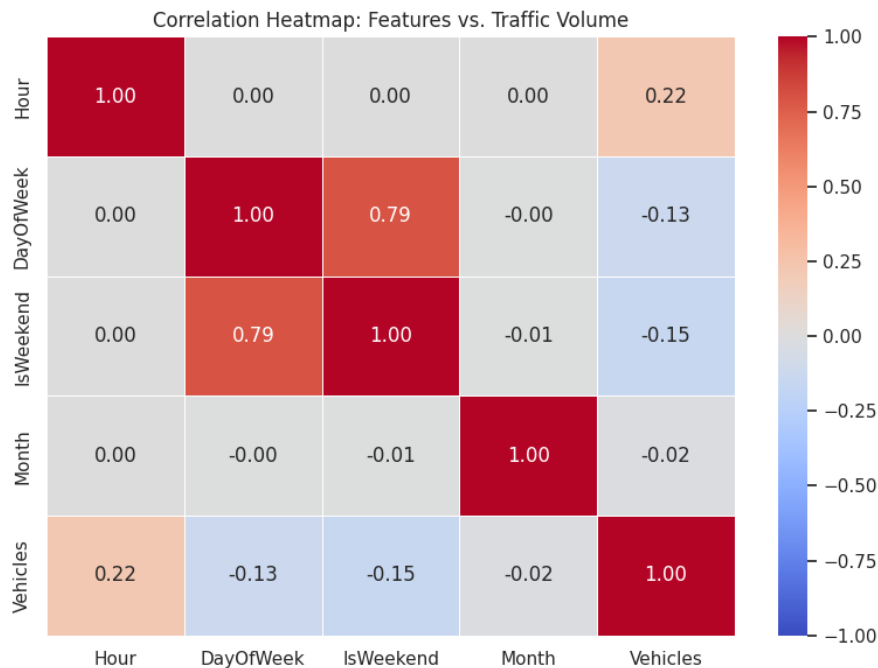


Figure 6: Correlation Heatmap — Features vs. Traffic Volume

Individual linear correlations between the engineered calendar features and vehicle count are modest (Hour: 0.22; DayOfWeek: -0.13; IsWeekend: -0.15), which is expected since traffic dynamics are driven by non-linear interactions between these variables rather than by any single linear relationship — exactly the kind of pattern that a tree-based ensemble like XGBoost is well suited to capture.

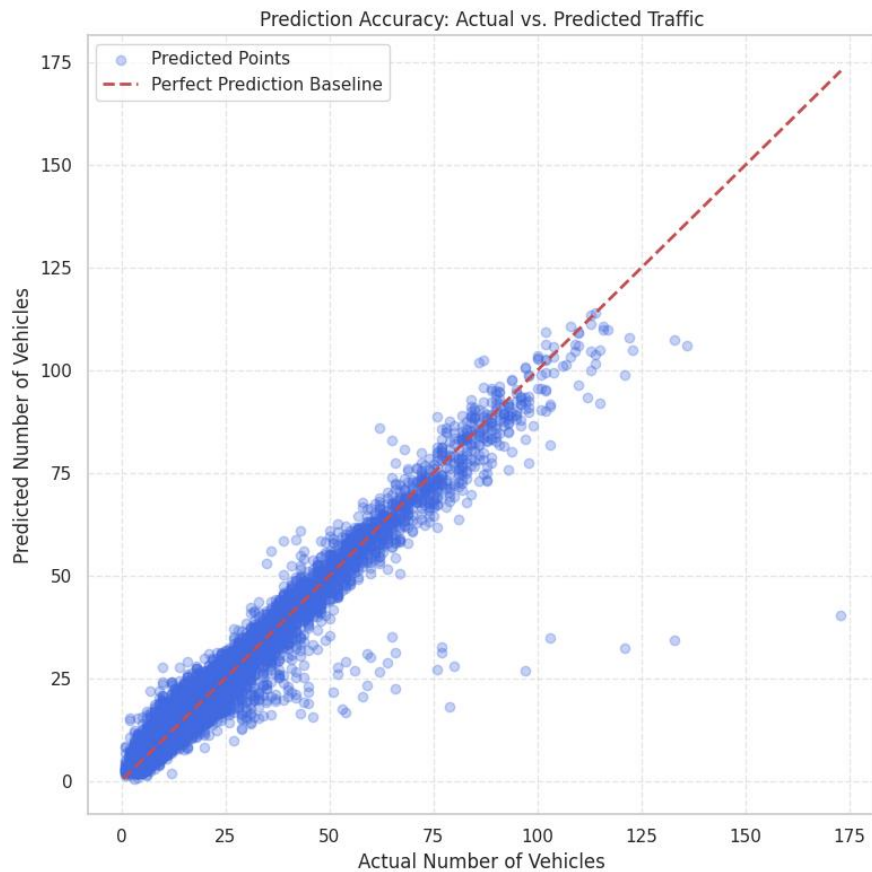


Figure 7: Prediction Accuracy — Actual vs. Predicted Traffic

The scatter plot shows the large majority of predictions clustering tightly around the perfect-prediction diagonal, particularly in the low-to-moderate traffic range where most of the data lies. A small number of extreme high-traffic outliers (above ≈ 150 vehicles/hour) are under-predicted, indicating that rare, unusually heavy-traffic events are the model's main remaining weakness.

7. My Learnings

This internship gave me my first end-to-end experience of turning a raw, messy, real-world dataset into a validated and interpretable machine learning solution within a fixed timeline, and the learnings extend well beyond just the modelling step itself.

- Real-world data is rarely clean: handling a doubly-nested ZIP archive with inconsistently named files taught me to build defensive, self-checking data-loading code (the fuzzy filename search) rather than assuming a fixed folder structure.
- Feature engineering matters more than model complexity: deriving Year, Month, Day, Hour, DayOfWeek, and IsWeekend from a single DateTime column contributed more to model performance than any hyperparameter change would have.
- Gradient boosting models like XGBoost can act as powerful, largely automatic feature-interaction learners — I did not need to manually construct an “Hour $\times$ IsWeekend” feature because the tree splits discovered that interaction on their own.
- A single well-designed metric is not enough: RMSE told me the average error magnitude, while R^2 told me how much of the underlying variance was actually explained — using both together, alongside visual diagnostics, gave me a far more honest picture of model quality than either alone.
- Diagnostic visualisation is a core modelling skill, not an afterthought: the feature-importance plot, the actual-vs-predicted scatter plot, and the correlation heatmap each revealed a different failure mode or strength of the model that a single accuracy number would have hidden.

- Respecting real-world constraints in post-processing (clipping negative predictions, rounding to whole vehicles) is a small step that is easy to forget but essential for a model's output to be usable downstream.

On the professional side, working independently against a fixed six-week deadline, documenting decisions clearly, and presenting results to a non-specialist audience are all skills directly transferable to any future AI/ML role. This project has also become a strong, concrete addition to my portfolio as I continue applying for AI/ML internships and roles.

8. Future Work Scope

Owing to the six-week time constraint of the internship, several promising directions could not be pursued but are recommended as future extensions of this work:

- Use a chronological (time-based) train/validation split instead of a random split, since adjacent hourly readings are highly autocorrelated — a chronological split would give a more realistic estimate of true forecasting performance on future, unseen periods.
- Incorporate exogenous features such as public holidays, local events, school calendars, and weather conditions, which are known to strongly influence traffic volume but were not available in the assigned dataset.
- Perform systematic hyperparameter tuning (e.g. grid search or Bayesian optimisation with Optuna) combined with time-series cross-validation for a more robust and reproducible choice of model settings.
- Benchmark the current XGBoost baseline against alternative approaches such as Facebook Prophet, LSTM/GRU sequence models, or a hybrid ARIMA + XGBoost-residual model.
- Investigate a hierarchical or transfer-learning approach for Junction 4, which has substantially less historical data (4,344 rows) than the other three junctions, to improve its “cold-start” forecasting quality.
- Package the trained model behind a lightweight REST API or dashboard for real-time inference, enabling integration with an actual smart-city traffic management system.
- Add an anomaly-detection layer to flag unusual spikes (e.g. accidents, festivals, road closures) that fall well outside the model's expected prediction range.